Coding Interview Patterns (Alex Xu & Shaun Gunawardane) – Interview Pattern Cheat Sheet

The book is organized around 19 major patterns that repeatedly appear in coding interviews. The goal is not memorizing solutions, but recognizing which pattern applies within the first 1–2 minutes of reading a problem. (ByteByteGo Blog)

---

1. Two Pointers

When to Use

- Sorted arrays
- Palindromes
- Pair problems
- In-place modifications

Core Idea

```
int left = 0;
int right = n - 1;

while(left < right) {
    ...
}
```

Variations

Opposite Direction

```
left -> <- right
```

Same Direction

```
slow -> fast ->
```

Common Problems

- Two Sum II
- Valid Palindrome
- Remove Duplicates
- Container With Most Water

Recognition Clues

- "sorted array"
- "pair"
- "remove duplicates"
- "palindrome"

---

2. Hash Maps & Sets

When to Use

Need O(1) lookup.

Core Idea

```
Map<K,V> map = new HashMap<>();
Set<T> set = new HashSet<>();
```

Common Uses

Frequency Counting

```
freq.put(x, freq.getOrDefault(x,0)+1);
```

Existence Check

```
if(set.contains(x))
```

Complement Lookup

```
target - nums[i]
```

Common Problems

- Two Sum
- Valid Anagram
- Contains Duplicate
- Group Anagrams

Recognition Clues

- duplicate
- frequency
- unique
- lookup

---

3. Linked Lists

Core Operations

Reverse

```
prev
curr
next
```

Merge

```
dummy node
```

Split

```
middle node
```

Common Problems

- Reverse Linked List
- Merge Lists
- Reorder List

Recognition Clues

- linked list manipulation
- constant space requirement

---

4. Fast & Slow Pointers

Core Idea

```
slow = slow.next;
fast = fast.next.next;
```

Applications

Cycle Detection

```
fast catches slow
```

Middle Node

```
slow reaches middle
```

Happy Number

Treat numbers as linked nodes.

Common Problems

- Linked List Cycle
- Happy Number
- Find Middle

Recognition Clues

- cycle
- middle element
- repeated state

---

5. Sliding Window

When to Use

Contiguous:

- subarray
- substring
- sequence

Fixed Window

```
window size = k
```

Variable Window

```
expand right
shrink left
```

Template

```
for(right...)
{
    add(right);

    while(invalid)
    {
        remove(left);
        left++;
    }
}
```

Common Problems

- Longest Substring Without Repeating Characters
- Maximum Sum Subarray
- Minimum Window Substring

Recognition Clues

- longest
- shortest
- contiguous

---

6. Binary Search

Classic Binary Search

```
while(low <= high)
```

Binary Search on Answer

Search answer space instead of array.

```
if(valid(mid))
    right = mid;
else
    left = mid + 1;
```

Common Problems

- Search Insert Position
- Koko Eating Bananas
- Capacity To Ship Packages

Recognition Clues

- sorted
- minimum possible
- maximum possible

---

7. Stack

When to Use

Need to remember recent elements.

Monotonic Stack

Increasing

```
while(stack.peek() > current)
```

Decreasing

```
while(stack.peek() < current)
```

Common Problems

- Daily Temperatures
- Next Greater Element
- Largest Rectangle

Recognition Clues

- nearest greater
- nearest smaller

---

8. Heap (Priority Queue)

Core Idea

Always access min/max efficiently.

```
PriorityQueue<Integer>
```

Common Problems

Top K

```
Top K Frequent
```

Merge K Lists

```
min heap
```

Streaming

```
Median Finder
```

Recognition Clues

- top k
- kth largest
- stream

---

9. Intervals

Core Idea

Sort first.

```
Arrays.sort(intervals)
```

Patterns

Merge

```
current overlaps previous
```

Sweep Line

Process starts and ends.

Common Problems

- Merge Intervals
- Meeting Rooms
- Insert Interval

Recognition Clues

- schedule
- overlap
- meeting

---

10. Prefix Sum

Core Formula

```
prefix[i]
```

Range:

```
sum(l,r)
=
prefix[r]-prefix[l-1]
```

Common Problems

- Range Sum Query
- Subarray Sum Equals K

Recognition Clues

- range sum

- many queries

---

11. Trees

DFS

Preorder

```
Root Left Right
```

Inorder

```
Left Root Right
```

Postorder

```
Left Right Root
```

BFS

```
Queue<TreeNode>
```

Common Problems

- Depth
- Diameter
- LCA
- Level Order

Recognition Clues

- hierarchy
- parent-child

---

12. Tries

Purpose

Efficient string lookup.

Structure

```
root
 ├─ a
 ├─ b
 └─ c
```

Operations

Insert

```
node.children[ch]
```

Search

```
walk character by character
```

Common Problems

- Implement Trie
- Word Search II
- Autocomplete

Recognition Clues

- dictionary
- prefix
- autocomplete

---

13. Graphs

Representation

Adjacency List

```
Map<Integer,List<Integer>>
```

Traversal

DFS

```
dfs(node)
```

BFS

```
queue
```

Common Problems

- Number of Islands
- Clone Graph
- Course Schedule

Recognition Clues

- network
- dependency
- connections

---

14. Backtracking

Core Formula

```
choose
explore
unchoose
```

Template

```
backtrack(path)
```

Common Problems

- Permutations
- Combinations
- N Queens
- Sudoku

Recognition Clues

- all possibilities
- generate all

---

15. Dynamic Programming

Recognition Test

Ask:

> Can the problem be broken into smaller overlapping subproblems?

Memoization

```
cache[state]
```

Tabulation

```
dp[i]
dp[i][j]
```

Common Patterns

Linear DP

```
House Robber
```

Grid DP

```
Unique Paths
```

Sequence DP

```
LCS
```

Knapsack

```
Take / Skip
```

Recognition Clues

- maximize
- minimize
- count ways

---

16. Greedy

Core Idea

Make the best decision now.

Template

```
sort
pick best
```

Common Problems

- Jump Game
- Gas Station
- Activity Selection

Recognition Clues

- local optimal
- interval scheduling

---

17. Sort & Search

Sorting

Merge Sort

```
divide
sort
merge
```

Quick Sort

```
partition
```

Search

Binary Search

Lower Bound

Upper Bound

Recognition Clues

- order matters
- rank
- kth element

---

18. Bit Manipulation

Check Bit

```
n & (1 << i)
```

Set Bit

```
n | (1 << i)
```

Clear Bit

```
n & ~(1 << i)
```

Toggle Bit

```
n ^ (1 << i)
```

XOR Pattern

```
a ^ a = 0
```

Common Problems

- Single Number
- Missing Number
- Power of Two

Recognition Clues

- O(1) space
- binary operations

---

19. Math & Geometry

Common Techniques

Modulo Arithmetic

```
(a+b)%m
```

GCD

```
gcd(a,b)
```

Coordinate Geometry

```
distance
slope
area
```

Matrix Simulation

```
rotate
traverse
transform
```

Common Problems

- Rotate Image
- Spiral Matrix
- Happy Number
- Pow(x,n)

Recognition Clues

- coordinates
- geometry
- math formulas

Alex Xu Pattern Recognition Table

Problem Clue	Pattern
Sorted array	Two Pointers / Binary Search
Duplicate detection	Hash Map / Set
Linked list cycle	Fast & Slow
Longest substring	Sliding Window
Top K	Heap
Overlapping meetings	Intervals
Range sum	Prefix Sum
Tree traversal	DFS/BFS
Prefix matching	Trie
Connected components	Graph DFS/BFS
Generate all possibilities	Backtracking
Optimal value/count ways	DP
Local optimal choice	Greedy
Kth/Lower Bound	Sort & Search
Binary representation	Bit Manipulation
Coordinate calculations	Math & Geometry

MAANG+ Interview Flow (Recommended)

When reading a problem, ask these questions in order:

```
1. Is it a HashMap/Set problem?
2. Is it sorted?
3. Is it a contiguous window?
4. Can two pointers help?
5. Is it a tree?
6. Is it a graph?
7. Do I need Top K?
8. Do I need all combinations?
9. Is this DP?
10. Can greedy work?
```

```
11. Can binary search reduce the answer space?
12. Is there a mathematical shortcut?
```

This book's pattern set is more structured and interview-focused than *Cracking the Coding Interview*. The 19 patterns effectively cover the majority of LeetCode Medium questions and a large portion of coding rounds at companies such as Google, Meta, Amazon, Microsoft, Apple, Uber, and Airbnb. The official table of contents lists these 19 chapters/patterns directly. (ByteByteGo Blog)

Coding Interview Patterns (Alex Xu & Shaun Gunawardane) – One-Page MAANG+ Pattern Reference

The book organizes interview problems into a set of reusable patterns. Compared to *Cracking the Coding Interview*, it is more pattern-centric and closely aligned with modern LeetCode-style interviews. The book covers 19 major patterns.

Pattern Selection Flow

```
Array/String?
├─ Sorted? → Two Pointers / Binary Search
├─ Contiguous? → Sliding Window
├─ Range Query? → Prefix Sum
├─ Top K? → Heap
├─ Duplicate/Frequency? → HashMap

Tree?
├─ Traversal? → DFS/BFS
├─ Parent/Ancestor? → Tree Pattern

Graph?
├─ Connectivity? → DFS/BFS
├─ Dependency? → Topological Sort
├─ Shortest Path? → BFS/Dijkstra

Need all possibilities?
└─ Backtracking
```

```
Optimization problem?
├─ Overlapping subproblems? → DP
└─ Local optimal works? → Greedy
```

---

1. Two Pointers

When to Use

- Sorted arrays
- Palindromes
- Pair problems
- In-place modifications

Template

```
int left = 0;
int right = n - 1;

while (left < right) {
    ...
}
```

Variants

Variant	Usage
Opposite Direction	Sorted arrays
Fast/Slow	In-place modifications
Multiple Pointers	Merge operations

Examples

- Two Sum II
- Valid Palindrome

- Remove Duplicates
- Container With Most Water

Complexity

```
Time: O(n)
Space: O(1)
```

---

2. Hash Maps & Sets

When to Use

Need:

```
Fast lookup
Frequency counting
Duplicate detection
```

Template

```
Map<K,V> map = new HashMap<>();
Set<T> set = new HashSet<>();
```

Subpatterns

Frequency Count

```
freq.put(x, freq.getOrDefault(x,0)+1);
```

Complement Search

```
target - nums[i]
```

Grouping

```
Map<String,List<String>>
```

Examples

- Two Sum
- Group Anagrams
- Contains Duplicate
- First Unique Character

Complexity

```
Time: O(n)
Space: O(n)
```

---

3. Linked Lists

Core Operations

Reverse

```
prev
curr
next
```

Merge

```
dummy node
```

Split

```
middle node
```

Examples

- Reverse Linked List
- Merge Two Lists
- Reorder List

---

4. Fast & Slow Pointers

Template

```
slow = slow.next;
fast = fast.next.next;
```

Applications

Problem Type	Usage
Cycle Detection	Floyd Algorithm
Middle Node	Single pass
Repeated State	Happy Number

Examples

- Linked List Cycle
- Happy Number
- Find Middle Node

Complexity

```
O(n)
```

---

5. Sliding Window

Recognition Clues

```
Longest
Shortest
Maximum
Minimum
Contiguous
Substring
Subarray
```

Fixed Window

```
window size = k
```

Variable Window

```
while(invalid) {
   shrink();
}
```

Generic Template

```
for(right...) {

   add(right);

   while(window invalid) {
      remove(left);
      left++;
   }
```

```
    updateAnswer();
}
```

Examples

- Longest Substring Without Repeating Characters
- Minimum Window Substring
- Maximum Average Subarray

Complexity

```
O(n)
```

---

6. Binary Search

Classic

```
while(low <= high)
```

Binary Search on Answer

```
if(valid(mid))
    right = mid;
else
    left = mid + 1;
```

Recognition Clues

```
Minimum possible
Maximum possible
Sorted
Monotonic condition
```

Examples

- Search Insert Position
- Koko Eating Bananas
- Capacity To Ship Packages

Complexity

```
O(log n)
```

---

7. Stack

When to Use

Need access to:

```
Most recent item
Previous greater/smaller element
```

Monotonic Stack

Increasing

```
while(stack.peek() > current)
```

Decreasing

```
while(stack.peek() < current)
```

Examples

- Daily Temperatures

- Next Greater Element
- Largest Rectangle in Histogram

Complexity

```
O(n)
```

---

8. Heap (Priority Queue)

Recognition Clues

```
Top K
Kth Largest
Median
Highest Priority
Stream
```

Template

```
PriorityQueue<Integer> pq;
```

Examples

- Top K Frequent Elements
- K Closest Points
- Find Median From Data Stream

Complexity

```
O(n log k)
```

---

9. Intervals

First Step

```
Arrays.sort(intervals);
```

Subpatterns

Merge

```
Current overlaps previous
```

Sweep Line

```
Process starts and ends
```

Examples

- Merge Intervals
- Insert Interval
- Meeting Rooms

Complexity

```
O(n log n)
```

---

10. Prefix Sum

Formula

```
prefix[i]
```

Range Query:

```
sum(l,r)
=
prefix[r] - prefix[l-1]
```

Examples

- Range Sum Query
- Subarray Sum Equals K

Complexity

```
Build: O(n)
Query: O(1)
```

---

11. Trees

DFS

Preorder

```
Root Left Right
```

Inorder

```
Left Root Right
```

Postorder

```
Left Right Root
```

BFS

```
Queue<TreeNode>
```

Examples

- Maximum Depth
- Diameter
- Lowest Common Ancestor
- Level Order Traversal

---

12. Trie

When to Use

```
Prefix
Dictionary
Autocomplete
Word Search
```

Template

```
class TrieNode {
    TrieNode[] children;
    boolean isWord;
}
```

Examples

- Implement Trie
- Word Search II
- Design Search Suggestions

Complexity

```
Insert: O(L)
Search: O(L)
```

---

13. Graphs

Representation

```
Map<Integer,List<Integer>>
```

Traversal

DFS

```
dfs(node)
```

BFS

```
queue.offer(node)
```

Examples

- Number of Islands
- Clone Graph
- Course Schedule

Complexity

```
O(V + E)
```

---

14. Backtracking

Core Formula

```
Choose
Explore
Unchoose
```

Template

```
backtrack(path) {

   if(solution)
      save();

   for(choice) {
      choose();
      backtrack();
      unchoose();
   }
}
```

Examples

- Permutations
- Combinations
- N Queens
- Sudoku Solver

---

15. Dynamic Programming

Recognition Test

Ask:

```
Can I express the answer
using smaller answers?
```

Core Formula

```
dp[state] =
   best(previous states)
```

Subpatterns

Pattern	Example
Linear DP	House Robber
Grid DP	Unique Paths
Sequence DP	LCS
Knapsack	Partition Equal Subset Sum

Examples

- Climbing Stairs
- Coin Change
- Edit Distance
- Longest Increasing Subsequence

---

16. Greedy

Core Idea

```
Take the best local choice
and never revisit.
```

Recognition Clues

```
Minimum intervals
Maximum activities
Reachability
```

Examples

- Jump Game
- Gas Station
- Non-overlapping Intervals

---

17. Sort & Search

Common Sorts

Sort	Complexity
Merge Sort	O(n log n)
Quick Sort	O(n log n) avg
Heap Sort	O(n log n)

Examples

- Kth Largest
- Merge Sorted Arrays
- Sort Colors

---

18. Bit Manipulation

Operations

Operation	Formula
Check Bit	n & (1<<i)
Set Bit	n
Clear Bit	n & ~(1<<i)
Toggle Bit	n ^ (1<<i)

Common Tricks

Power of Two

```
(n & (n-1)) == 0
```

Single Number

```
a ^ a = 0
```

Examples

- Single Number
- Missing Number
- Counting Bits

---

19. Math & Geometry

Common Topics

GCD

```
gcd(a,b)
```

Modulo

```
(a+b)%m
```

Matrix Operations

```
rotate
transpose
spiral
```

Coordinate Geometry

```
distance
slope
area
```

Examples

- Rotate Image
- Spiral Matrix
- Pow(x,n)

MAANG+ Interview Recognition Table

If You See...	Think...
Sorted Array	Two Pointers / Binary Search
Duplicate Detection	HashMap
Longest Substring	Sliding Window
Top K Elements	Heap

If You See...	Think...
Overlapping Meetings	Intervals
Range Sum	Prefix Sum
Parent-Child Structure	Tree
Prefix Search	Trie
Connected Components	Graph DFS/BFS
Generate All Solutions	Backtracking
Count Ways / Min Cost	DP
Local Best Choice	Greedy
Kth Element	Sort & Search
Binary Constraints	Bit Manipulation
Coordinates / Matrix	Math & Geometry

The 10 Patterns That Solve ~80–90% of Interview Questions

If you're targeting Google, Meta, Amazon, Microsoft, Apple, Uber, Airbnb, Stripe, Databricks, Rubrik, etc., master these first:

1. HashMap / Set
2. Two Pointers
3. Sliding Window
4. Binary Search
5. Tree DFS/BFS
6. Graph DFS/BFS
7. Heap (Top K)
8. Backtracking
9. Dynamic Programming
10. Intervals

Most medium-level interview questions are combinations of these patterns, with DP and Graphs being the most challenging areas. The Alex Xu book explicitly organizes interview preparation around recognizing and applying these recurring patterns.